

Python List Slicing & Indexing — Pre-Lab Study Guide

(with a bridge into Genetic Algorithm structure)

1. Indexing — the fast refresher

python

```
lst = [10, 20, 30, 40, 50]
```

```
lst[0]    # 10  -> first element
lst[-1]   # 50  -> last element
lst[2]    # 30  -> third element
lst[-2]   # 40  -> second to last
```

Rules that matter:

- Indices go `0` to `len(lst)-1` forward, `-1` to `-len(lst)` backward.
 - `lst[i]` where `i` is out of range → **raises** `IndexError`. This is the #1 difference from slicing (see below).
 - Assignment works the same way: `lst[0] = 99` replaces a single element in place.
-

2. Slicing — the syntax you actually need to internalize

python

```
lst[start:stop:step]
```

- `start` — inclusive (default `0`)
- `stop` — exclusive (default `len(lst)`)
- `step` — default `1`

python

```

lst = [0,1,2,3,4,5,6,7,8,9]

lst[2:5]      # [2,3,4]      indices 2,3,4
lst[:4]       # [0,1,2,3]    start defaults to 0
lst[6:]       # [6,7,8,9]    stop defaults to len
lst[:]        # full shallow copy
lst[::-2]     # [0,2,4,6,8]  every 2nd element
lst[1::2]     # [1,3,5,7,9]  every 2nd, starting at 1
lst[::-1]     # [9,8,7,6,5,4,3,2,1,0]  reversed
lst[-3:]      # [7,8,9]      last 3
lst[:-3]      # [0,1,2,3,4,5,6]  all but last 3

```

Critical gotcha: slicing never raises `IndexError`, even with wild out-of-bounds numbers:

```

python

lst[100:200]  # []    -- no crash, just empty
lst[-100:3]   # [0,1,2] -- clips silently to valid range

```

That silent-clipping behavior is genuinely useful (you don't need bounds-checking before a slice), but it also hides bugs — if you expect 5 elements back and get 2, the slice didn't error, your math was wrong.

3. Power tricks (the "expert" toolkit)

Reverse without `reversed()`

```

python

lst[::-1]

```

Shallow copy (independent list, same nested objects)

```

python

new_lst = lst[:]      # or lst.copy()

```

Swap two elements

```

python

a[i], a[j] = a[j], a[i]

```

python

```
lst[0:2], lst[2:4] = lst[2:4], lst[0:2]
```

Insert without `.insert()` — slice assignment can grow the list:

python

```
lst[2:2] = [99, 100]      # inserts at index 2, nothing removed
```

Delete a range

python

```
del lst[2:5]
```

Replace a range with a different-length chunk (slices don't have to match size!)

python

```
lst[1:3] = [100]          # replaces 2 elements with 1 -> list shrinks
```

Chunk a list into equal pieces

python

```
n = 3
chunks = [lst[i:i+n] for i in range(0, len(lst), n)]
```

Rotate a list left by k (pure slicing, no loop)

python

```
k = 2
rotated = lst[k:] + lst[:k]
```

Star-unpacking (technically indexing's cousin)

python

```
first, *middle, last = lst
```

Watch this trap — `list * n` with nested lists shares references:

python

```
grid = [[0]*3] * 3          # BAD: all 3 rows are the SAME list object
grid[0][0] = 1              # mutates every row!

grid = [[0]*3 for _ in range(3)] # correct: independent rows
```

4. Practice Problems (do these before checking solutions)

Use this base list for all problems unless stated otherwise:

python

```
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

1. Get the first and last elements using two different indexing expressions each (4 expressions total).
2. Get the **last 4 elements** using slicing — without calling `len()`.
3. Reverse `lst` using slicing only.
4. Get every 3rd element starting from index 1.
5. Split `lst` into two halves, `left` and `right`. Make your function work correctly even when the list length is **odd**.
6. Write `swap_halves(lst)` that swaps the first half and second half of a list using slice assignment.
7. Given `lst2 = list(range(9))`, remove the middle 3 elements (`del` + slicing) so you're left with `[0,1,2,3,4,7,8]`... wait, work out what indices that actually requires.
8. Write `rotate_left(lst, k)` that rotates a list left by `k` positions, using only slicing (no explicit loop).
9. Write `chunk_list(lst, n)` that returns sublists of length `n` (last chunk may be shorter).
10. **GA warm-up:** write `single_point_crossover(p1, p2)` — two equal-length lists in, a random crossover point chosen, two children returned by slicing and recombining `p1`/`p2` at that point.

5. Solutions

python

```

# 1.
lst[0], lst[-len(lst)]      # first
lst[-1], lst[len(lst)-1]    # last

# 2.
lst[-4:]

# 3.
lst[::-1]

# 4.
lst[1::3]

# 5.
def split_half(lst):
    mid = len(lst) // 2
    return lst[:mid], lst[mid:]    # right half absorbs the extra element if odd

# 6.
def swap_halves(lst):
    mid = len(lst) // 2
    lst[:mid], lst[mid:] = lst[mid:], lst[:mid]
    return lst

# 7. To remove the 3 *middle* elements of a 9-element list (indices 0-8),
#     middle = indices 3,4,5
lst2 = list(range(9))
del lst2[3:6]                # -> [0,1,2,6,7,8]

# 8.
def rotate_left(lst, k):
    k = k % len(lst)          # handle k > len(lst)
    return lst[k:] + lst[:k]

# 9.
def chunk_list(lst, n):
    return [lst[i:i+n] for i in range(0, len(lst), n)]

# 10.
import random
def single_point_crossover(p1, p2):
    point = random.randint(1, len(p1) - 1)
    child1 = p1[:point] + p2[point:]

```

```
child2 = p2[:point] + p1[point:]
return child1, child2
```

6. Why this connects directly to your GA lab

A genetic algorithm represents each candidate solution (a **chromosome**) as a list of genes. Almost every core operator is just indexing/slicing on those lists:

GA Concept	What it actually is in Python
Chromosome	A <code>list</code> of genes (numbers, bits, or symbols)
Population	A <code>list</code> of chromosomes (list of lists)
Fitness evaluation	Iterating/indexing over a chromosome's genes
Single-point crossover	<code>parent1[:point] + parent2[point:]</code>
Two-point crossover	<code>p1[:pt1] + p2[pt1:pt2] + p1[pt2:]</code>
Mutation	Index assignment: <code>chromosome[i] = new_gene</code>
Elitism / selection	Slicing the top-k from a sorted population list

7. Standard Genetic Algorithm skeleton in Python

```
python
```

```

import random

# ---- 1. Representation ----
def create_individual(length, gene_pool):
    return [random.choice(gene_pool) for _ in range(length)]

def create_population(pop_size, length, gene_pool):
    return [create_individual(length, gene_pool) for _ in range(pop_size)]

# ---- 2. Fitness ----
def fitness(individual, target):
    return sum(1 for gene, t in zip(individual, target) if gene == t)

# ---- 3. Selection (tournament) ----
def tournament_selection(population, fitnesses, k=3):
    contenders = random.sample(list(zip(population, fitnesses)), k)
    contenders.sort(key=lambda pair: pair[1], reverse=True)
    return contenders[0][0]

# ---- 4. Crossover ----
def single_point_crossover(p1, p2):
    point = random.randint(1, len(p1) - 1)
    return p1[:point] + p2[point:], p2[:point] + p1[point:]

def two_point_crossover(p1, p2):
    pt1, pt2 = sorted(random.sample(range(1, len(p1)), 2))
    child1 = p1[:pt1] + p2[pt1:pt2] + p1[pt2:]
    child2 = p2[:pt1] + p1[pt1:pt2] + p2[pt2:]
    return child1, child2

# ---- 5. Mutation ----
def mutate(individual, mutation_rate, gene_pool):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            individual[i] = random.choice(gene_pool)
    return individual

# ---- 6. Main loop ----
def genetic_algorithm(target, pop_size, gene_pool, generations,
                      mutation_rate=0.01, elitism=2):
    length = len(target)
    population = create_population(pop_size, length, gene_pool)

```

```

for gen in range(generations):
    fitnesses = [fitness(ind, target) for ind in population]

    # elitism: carry the best `elitism` individuals straight through
    ranked = sorted(zip(population, fitnesses), key=lambda p: p[1], reverse=True)
    new_population = [ind for ind, _ in ranked[:elitism]]

    while len(new_population) < pop_size:
        parent1 = tournament_selection(population, fitnesses)
        parent2 = tournament_selection(population, fitnesses)
        child1, child2 = single_point_crossover(parent1, parent2)
        new_population.append(mutate(child1, mutation_rate, gene_pool))
        new_population.append(mutate(child2, mutation_rate, gene_pool))

    population = new_population[:pop_size]

    best = max(population, key=lambda ind: fitness(ind, target))
    if fitness(best, target) == length:
        print(f"Solved at generation {gen}")
        break

return max(population, key=lambda ind: fitness(ind, target))

```

Run it as a sanity check:

```

python

target = [1,0,1,1,0,1,0,0,1,1]
best = genetic_algorithm(target, pop_size=50, gene_pool=[0,1], generations=200)
print(best)

```

The whole loop is: **make population** → **score it** → **pick parents** → **slice them into children** → **flip a few genes** → **repeat**. Every one of those verbs is a list operation you just practiced above.

9. Dictionaries — indexing & access patterns

Unlike lists, dictionaries are key-based, not position-based. There's no numeric index, and dictionaries **cannot be sliced** — `d[1:3]` raises `TypeError: unhashable type: 'slice'`, because slicing tries to look up a key literally called `slice(1, 3)`.

```
python
```



```
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}

d['c']          # 3          -- direct key access, raises KeyError if missing
d.get('c')      # 3          -- safe access
d.get('z')      # None       -- no crash
d.get('z', 0)   # 0          -- custom default
```

Since Python 3.7+, dicts preserve insertion order, so "slicing-like" behavior is possible by going through `.items()`, `.keys()`, or `.values()` and converting to a list first:

```
python

list(d.items())[:3]    # first 3 key-value pairs as a list of tuples
list(d.keys())[-2:]    # last 2 keys
list(d.values())[::2]  # every other value
```

10. Dict tricks

- swap keys/values: `{v: k for k, v in d.items()}`
- merge: `d1 | d2` (3.9+) or `{**d1, **d2}`
- build from two parallel lists: `dict(zip(keys, values))`
- filter by condition: `{k: v for k, v in d.items() if v > 2}`
- sort by value: `dict(sorted(d.items(), key=lambda item: item[1]))`
- nested access without crashing: `d.get('a', {}).get('b')` instead of `d['a']['b']`
- count occurrences in a list: `from collections import Counter; Counter(some_list)`
- group items into buckets: `from collections import defaultdict`

11. Dict practice problems

1. Given `d = {'a':5, 'b':2, 'c':8, 'd':1, 'e':9}`, get the value for key `'c'` two different ways.
2. Access a key that doesn't exist — once in a way that crashes, once in a way that doesn't.
3. Get the first 2 and last 2 key-value pairs as lists (remember: no direct slicing on dicts).
4. Invert `d` so values become keys and keys become values.
5. Write a dict comprehension that keeps only entries where the value is greater than 4.

6. Sort `d` by value, descending, returning a new dict in that order.
7. Given `keys = ['x', 'y', 'z']` and `values = [10, 20, 30]`, build a dict in one line.
8. Merge `d1 = {'a': 1, 'b': 2}` and `d2 = {'b': 99, 'c': 3}` so that `d2`'s values win on conflicts.

Solutions:

```
python

# 1.
d['c']
d.get('c')

# 2.
d['z']          # raises KeyError -- crashes
d.get('z')      # returns None -- safe

# 3.
items = list(d.items())
items[:2]       # first 2
items[-2:]     # last 2

# 4.
{v: k for k, v in d.items()}

# 5.
{k: v for k, v in d.items() if v > 4}

# 6.
dict(sorted(d.items(), key=lambda item: item[1], reverse=True))

# 7.
dict(zip(keys, values))

# 8.
{**d1, **d2}    # or d1 | d2 in Python 3.9+
```

12. List + Dictionary combined problems — the GA-relevant pattern

A very common, arguably cleaner, way to represent a GA population is a **list of dictionaries**, where each dict holds an individual's genes plus its fitness:

python

```
population = [  
    {'genes': [1,0,1,1,0], 'fitness': 3},  
    {'genes': [0,1,1,0,0], 'fitness': 2},  
    {'genes': [1,1,1,1,1], 'fitness': 5},  
]
```

This pattern shows up constantly once labs move past "just lists," so it's worth being fluent switching between the two structures.

Problems:

1. From `population` above, extract just the `genes` lists into a plain list of lists.
2. Sort `population` by `'fitness'`, descending.
3. Get the **top 2** fittest individuals (combine sorting + slicing).
4. Write `average_fitness(population)` that returns the mean fitness across the population.
5. Write a function that builds a population of dicts from a list of gene-lists, computing fitness for each via a `fitness()` function.
6. Group individuals into a dict bucketed by fitness value, e.g. `{5: [individual, ...], 3: [...], ...}`.
7. Write `crossover_population(population)` that pairs up individuals two at a time (stepping through the list 2 at a time) and performs single-point crossover on their `'genes'` lists, returning a new list of child dicts (fitness left as `None` for now).

Solutions:

python

```
# 1.
gene_lists = [ind['genes'] for ind in population]

# 2.
ranked = sorted(population, key=lambda ind: ind['fitness'], reverse=True)

# 3.
top2 = sorted(population, key=lambda ind: ind['fitness'], reverse=True)[:2]

# 4.
def average_fitness(population):
    return sum(ind['fitness'] for ind in population) / len(population)

# 5.
def build_population(gene_lists, fitness_fn):
    return [{'genes': genes, 'fitness': fitness_fn(genes)} for genes in gene_lists]

# 6.
from collections import defaultdict
def group_by_fitness(population):
    buckets = defaultdict(list)
    for ind in population:
        buckets[ind['fitness']].append(ind)
    return dict(buckets)

# 7.
import random
def crossover_population(population):
    children = []
    for i in range(0, len(population) - 1, 2): # step 2 at a time
        parent1, parent2 = population[i], population[i + 1]
        point = random.randint(1, len(parent1['genes']) - 1)
        child1_genes = parent1['genes'][:point] + parent2['genes'][point:]
        child2_genes = parent2['genes'][:point] + parent1['genes'][point:]
        children.append({'genes': child1_genes, 'fitness': None})
        children.append({'genes': child2_genes, 'fitness': None})
    return children
```

13. Quick Cheat Sheet

```
lst[i]           single element, can IndexError
lst[i:j]         sublist [i, j), never errors
lst[i:j:k]       every kth element from i to j
lst[::-1]        reverse
lst[:]           shallow copy
lst[a:b] = [...] replace a range (can change length)
del lst[a:b]      remove a range
```

```
d[key]           value at key, KeyError if missing
d.get(key, default) safe access, no crash
d[1:3]           INVALID -- TypeError, dicts can't be sliced
list(d.items())[:n] "first n" workaround via list conversion
{**d1, **d2}      merge (d2 wins on conflicting keys)
{v:k for k,v in d.items()} invert a dict
```